JAVA

Easy Level

1. **What is Java**
   Java is a programing language, it is a high-level object-oriented programing language, it is designed to be platform independent which means that we can run java code on any devices which has JVM.

2. **What are main features of Java**
   The main features of java including
   - Platform independence
   - Object-oriented
   - Simple and easy to learn
   - Secure
   - Robust
   - High performance

3. **Explain the concept of OPP's**
   Opps is a programing paradigm based on the concept of objects. It allows for the creation of classes, which are blueprints for objects. The four main principles of Opps are encapsulation, inheritance, polymorphism and abstraction.

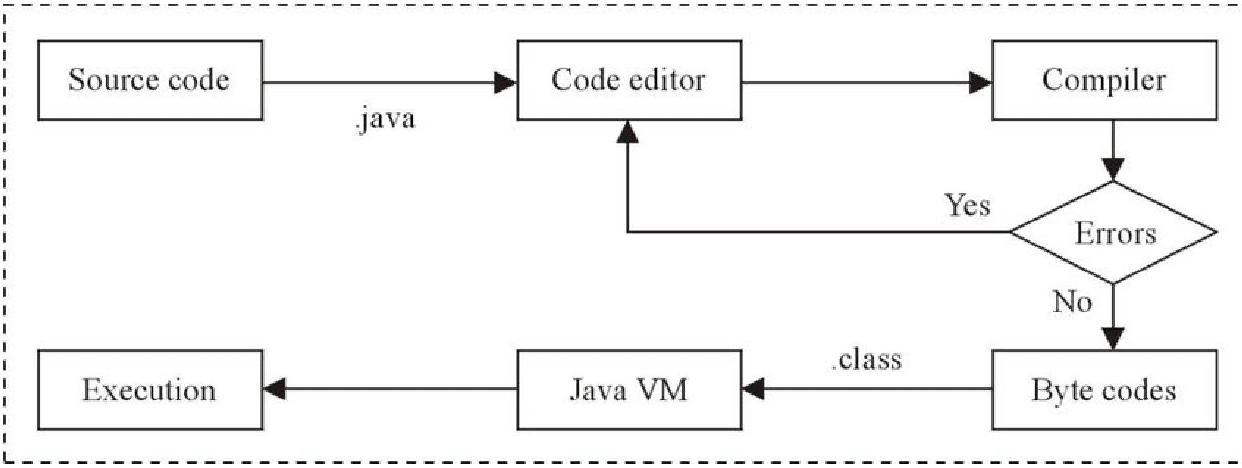
4. **What is Class in Java**
   A class in java is a blueprint to create objects that shares common properties and methods

5. **What is an Object in Java?**
   An Object is an instance of class. It is a concrete entity that has state (attribute) and behavior (methods) as defined by its class.

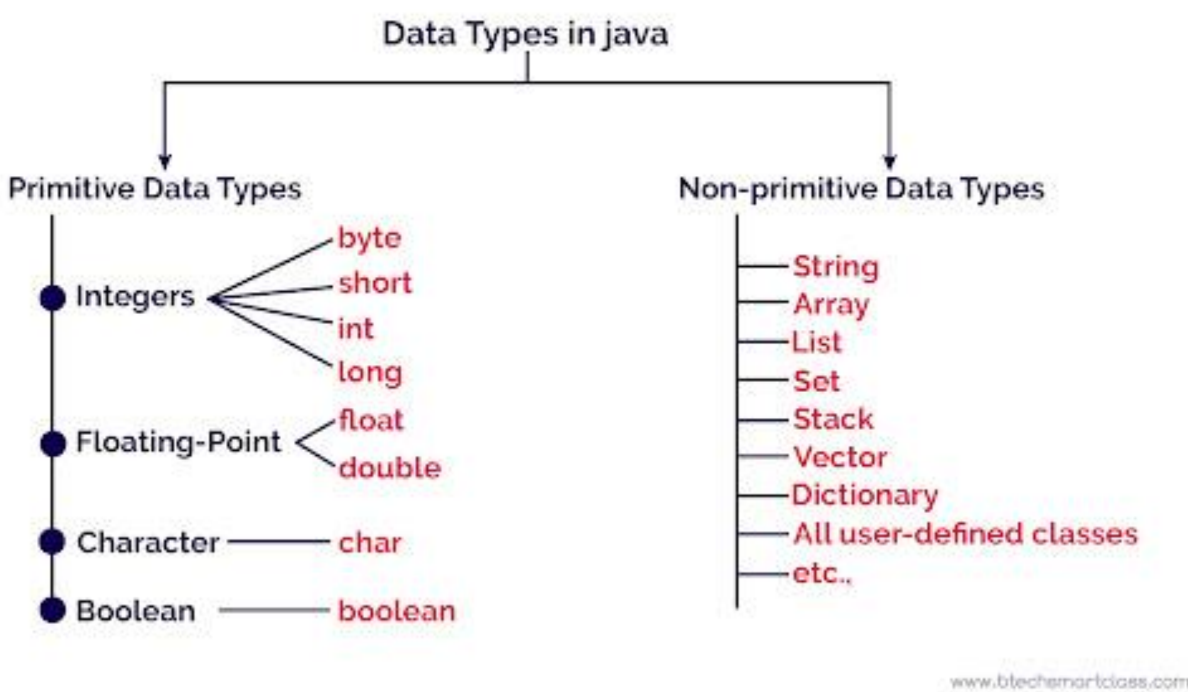
6. **What is difference between JDK, JRE and JVM**
   - **JDK (Java Development Kit): -** It is a software development environment used for developing Java applications. It includes JRE and java development tools
   - **JRE (Java Runtime Environment): -** It provides the libraries, Java virtual Machines (JVM) and other components to run code written in java

- **JVM (Java Virtual Machine): -** It is an abstract machine that enables your computer to run java program. It converts java byte code into machine language.

Execution of Java code: -



7. What are different data types in Java



7. **What is constructor in Java**

   In Java a constructor is like a method. The property of a constructor is that it must have the same name as per the class name and it has no return type. We don't need to call the constructor manually; it automatically invokes implicitly during the installation.

Ex: -

```
public class Person {
  String name;
  int age;

  // Constructor
  public Person(String name, int age) {
    this.name = name;
    this.age = age;
  }
}
```

8. **What is polymorphism and explain it briefly?**

Real-life Illustration of Polymorphism in Java: A person at the same time can have different characteristics. Like a man at the same time is a father, a husband, and an employee. So, the same person possesses different behaviors in different situations. This is called polymorphism

It is a concept in Java which we can perform a single action in different ways.
The term polymorphism derived from 2 Greek words

Poly — many
Morphs — forms

Method Overloading: -

Method Overloading is the process in which the class has two or more methods with the same name. Nevertheless, the implementation of a specific method occurs according to the number of parameters in the method call.

java

```java
class MathOperations {
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }
}

public class Main {
    public static void main(String[] args) {
        MathOperations math = new MathOperations();
        System.out.println(math.add(5, 3));     // Outputs: 8
        System.out.println(math.add(5.5, 3.2)); // Outputs: 8.7
    }
}
```

Method Overriding: -

Method Overriding is a procedure in which the compiler can allow a child class to implement a specific method already provided in the parent class.

(OR)

When a subclass provides its own version of a method that is already defined in its superclass, it's called method overriding. This means that the subclass can customize how the method works while keeping the same method name and parameters.

java Copy code

```java
class Animal {
    void makeSound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void makeSound() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal myDog = new Dog();
        myDog.makeSound(); // Outputs: Dog barks
    }
}
```

9. **What is Inheritance in Java**

Inheritance in Java is a mechanism in which one class acquires all the properties and behaviors of a parent class. This promotes code reusability and establishes a hierarchy between classes.
"**extends**" is a key word which implements the inheritance concept.2

10. **What is encapsulation in Java?**
Encapsulation in Java is a process of wrapping data (fields) and code (methods) together as a single unit. **(OR)** Hiding the internal state of an object and only exposing necessary functionalities through methods

Ex: -

```java
public class LoginPage {
    // Web elements are encapsulated as private fields
    private WebDriver driver;
    private By usernameField = By.id("username");
    private By passwordField = By.id("password");
    private By loginButton = By.id("loginButton");

    // Constructor
    public LoginPage(WebDriver driver) {
        this.driver = driver;
    }

    // Public methods to interact with the elements
    public void enterUsername(String username) {
        driver.findElement(usernameField).sendKeys(username);
    }

    public void enterPassword(String password) {
        driver.findElement(passwordField).sendKeys(password);
    }

    public void clickLogin() {
        driver.findElement(loginButton).click();
    }
}
```

11. **What is Abstraction in Java?**

Abstraction in Java is the concept of hiding the complex implementation details of the system and exposing only the necessary part to the user
Ex: - when we withdrawal money from the ATM we can withdrawl the money but unable to see the internal implementation

There are two ways to achieve abstraction in Java

- Abstract Class (0 to 100%)
- Interfaces (100%)

Abstract class in Java

A class declared with the abstract keyword is known as an abstract class in Java.

It can have both Abstract and non-Abstract methods. It cannot be instantiated.

POINTS TO REMEMBER: -

- An Abstract class must be declared with "**Abstract**" key word
- It can have Abstract and non- Abstract methods
- It cannot be instantiated (we cannot create direct object for the Abstract class)
- It can have final methods
- It can have constructors and static methods also

```
abstract class Bike{
 abstract void run();
}
class Honda4 extends Bike{
void run(){System.out.println("running safely");}
public static void main(String args[]){
 Bike obj = new Honda4();
 obj.run();
}
}
```

Test it Now

```
running safely
```

Interfaces in Java

An Interface in Java is a blueprint of a class. It has static constants and abstract methods.

Why use Java Interfaces

- It is used to achieve 100% abstraction
- By interface we can support the multiple inheritance
- It can be used to achieve loose coupling

How to declare interfaces

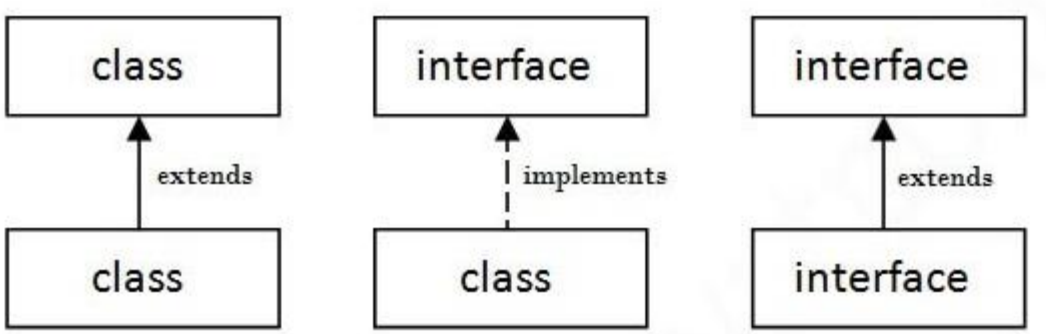
Syntax:

```
interface <interface_name>{

    // declare constant fields
    // declare methods that abstract
    // by default.
}
```

The relationship between Class and Interfaces

As shown in the below figure, A class extends another class, An Interface extends another Interface but a **Class Implements an Interface**



Java Interface Example: -

```java
interface printable{
void print();
}
class A6 implements printable{
public void print(){System.out.println("Hello");}

public static void main(String args[]){
A6 obj = new A6();
obj.print();
 }
}
```

Test it Now

Output:

```
Hello
```

Usage in Selenium Java:

In Selenium Java, abstraction can be used to create a high-level design of the framework where the implementation details are hidden. This is typically achieved through the Page Object Model (POM) and interface-driven design.

```java
public interface LoginPage {
    void enterUsername(String username);
    void enterPassword(String password);
    void clickLogin();
}

public class LoginPageImpl implements LoginPage {
    private WebDriver driver;
    private By usernameField = By.id("username");
    private By passwordField = By.id("password");
    private By loginButton = By.id("loginButton");

    public LoginPageImpl(WebDriver driver) {
        this.driver = driver;
    }

    @Override
    public void enterUsername(String username) {
        driver.findElement(usernameField).sendKeys(username);
    }

    @Override
    public void enterPassword(String password) {
        driver.findElement(passwordField).sendKeys(password);
    }

    @Override
    public void clickLogin() {
        driver.findElement(loginButton).click();
    }
}
```

12. **What is the difference between Abstract Class and Interface?**
    An abstract class can have method implementations, instance variables, and constructors, while an interface can only have method signatures and static final variables. Classes can extend only one abstract class but can implement multiple interfaces.
13. **What are access modifiers in Java**

Access modifiers in Java are keywords used to set the accessibility of classes, methods, and other members. The four access modifiers are:

- **public**: accessible from any other class
- **protected**: accessible within the same package and subclasses
- **default** (no modifier): accessible only within the same package
- **private**: accessible only within the same class

	Default	Private	Protected	Public
Same Class	Yes	Yes	Yes	Yes
Same Package Subclass	Yes	No	Yes	Yes
Same Package Non-Subclass	Yes	No	Yes	Yes
Different Package Subclass	No	No	Yes	Yes
Different Package Non-Subclass	No	No	No	Yes

In simple

- **Public:** Accessible from any class, anywhere.
- **Private:** Accessible only within the same class.
- **Default (Package-private):** Accessible within the same package.
- **Protected:** Accessible within the same package and by subclasses (if we inherited the protected class to this class) in different packages.

14. **What is final keyword in Java**

    The **FINAL** key word is used to denote constants. It can be applied to classes, methods and variables

    - A final class cannot be sub classed.
    - A final class cannot be overridden by subclass
    - A final variable cannot be changed, once it is initialized.

15. **What is Static keyword in Java**

The static keyword in java is used for memory management mainly. We can apply static keyword with variables, methods, blocks and nested classes.

Key Points:

- **Class Level**: Static methods belong to the class, not to any specific instance of the class.
- **Call Without Instance**: Can be called without creating an object of the class.
- **Static Context**: Can access other static methods and static variables directly.
- No **this** Keyword: Cannot use the **this** keyword because there is no instance context.
- **Utility Functions**: Often used for utility or helper methods that perform operations not dependent on instance variables.

16. **What is the difference between static and instance methods?**

Static methods belong to the class and can be called without creating an object, whereas instance method belongs to instance of the class and require an object to be called.

17. **What is "this" key word in Java?**

The "**this**" keyword is used to refer to the current instance of the class. It is often used to resolve the conflicts between instance variables and parameters with the same name.

18. **What is the 'super' keyword in Java?**

The super keyword is used to refer to the immediate parent class object. It is used to access parent class methods and constructors.

19. **What is the difference between String, StringBuilder, and StringBuffer?**

String is immutable, meaning its value cannot be changed once created. StringBuilder and StringBuffer are mutable and can be modified. StringBuilder is not synchronized, while StringBuffer is synchronized and thread-safe.

20. **How do you create array in Java?**

- Int [] numbers = new int [5]
- Int [] moreNumbers = {1,2,4,6,7,35}

21. **What is the purpose of the main method in Java?**
    The main method is the entry point of any java application. It is a static method and takes array of string as arguments and it will return void.

22. **What is a package in Java?**
    A package is a namespace that organizes classes and interfaces. It helps avoid name conflicts and provides controlled access. Packages are declared using the package keyword.

23. **What is the import statement in Java?**
    The import statement allows the programmer to use classes and interfaces from other packages. It makes it easier to refer to them without using their fully qualified names.

24. **What is the difference between break and continue statements in Java?**
    The `break` statement terminates the closest enclosing loop or switch statement, while the `continue` statement skips the current iteration and proceeds to the next iteration of the closest enclosing loop.

25.